

Harness Engineering: A Practical Handbook

Case study: a banking AI agent, built in Python with Google ADK, LiteLLM, and OpenAI *gpt-4o-mini*.

This document explains **what Harness Engineering is**, and walks through a real, working example of it being applied one layer at a time. The banking use case is not the point — a chatbot that checks balances and moves money is a convenient, easy-to-reason-about vehicle for demonstrating something more general: *how you build reliable systems around a fundamentally unreliable component (an LLM)*.

Part 1 — What Is Harness Engineering?

An AI agent is, at its core, a language model that can call tools. The model alone is a probabilistic text generator: it does not guarantee correctness, does not enforce rules, and cannot be trusted to reliably refuse a bad instruction, correctly remember a constraint, or accurately report what actually happened after it acts. **Harness Engineering is the discipline of building deterministic, non-LLM software around that model so the system behaves reliably even though the model doesn't.**

The model contributes judgment, language understanding, and tool selection. The harness contributes everything the model cannot be trusted to guarantee on its own. Harness engineering for AI agents spans several largely independent concerns:

#	Aspect	What it's about	Covered in this project?
1	Context management	What goes into the prompt/context window — instructions, retrieved documents, conversation history, truncation strategy	✗ Not covered
2	Memory	What the agent remembers across turns or sessions — short-term working memory, long-term persistent memory, what gets forgotten	✗ Not covered (all state is in-memory for one session)
3	Tool / action governance	Deciding, in code, whether a requested action is actually allowed — not just hoping the model asked correctly	✓ Step 2
4	Human-in-the-loop / confirmation	Requiring an explicit, verifiable approval step before a consequential action executes	✓ Step 3
5	Outcome verification	Checking what <i>actually</i> happened after an action, instead of assuming a tool call returning means it succeeded	✓ Step 4
6	Execution limits / runaway protection	Bounding retries, tool-call chains, and repeated failures so a confused or misbehaving agent can't loop forever	✓ Step 5

#	Aspect	What it's about	Covered in this project?
7	Observability / audit trails	Making the agent's decisions inspectable — for debugging, trust, and compliance	✅ Step 5
8	Multi-agent orchestration	Decomposing work across specialized agents, routing between them	❌ Not covered (one agent, by design)
9	Evaluation & testing	Systematically measuring agent quality against test cases	❌ Not covered (explicitly out of scope for this POC)
10	Security / prompt-injection defense	Protecting against malicious input or poisoned tool output steering the agent	❌ Not covered (adjacent to #3, but not the focus)
11	Cost / latency management	Model routing, caching, batching	❌ Not covered

This project deliberately isolates **rows 3–7** — the harness around a single agent's *actions* — and builds it up one deterministic layer at a time, in front of a live audience/reader, so each layer's necessity is obvious from the failure it fixes. Rows 1, 2, 8, 9, 10, and 11 are real and important, but they're different problems with different tools; folding them in would dilute the one thing this POC is trying to teach clearly.

Part 2 — The Use Case

A single Google ADK agent (`app/agent.py`), backed by OpenAI `gpt-4o-mini` through LiteLLM, with five tools operating on in-memory mock data (`app/mock_data.py`):

- `get_balance` — read the (one) savings account.
- `find_beneficiaries` — look up saved payees.
- `initiate_transfer` — validate and stage a money transfer.
- `confirm_transfer` — execute a staged transfer.
- `get_transaction_status` — look up a past transfer's status.

There's no database, no multiple agents, and no test suite — this is a demonstration artifact, not a product. Everything that matters happens in five sequential steps, each one refactoring the same codebase rather than adding a parallel version. **Step 1 is deliberately unguarded** — it exists to show the failure mode that every subsequent step fixes.

Step 1 — The Baseline: No Harness

The pain point

Give an LLM tool access and a system prompt, and it will do a *plausible-sounding* job — right up until it doesn't. In Step 1, `transfer_money` was a single tool that resolved a beneficiary by a loose name match and executed immediately:

- Ask to "transfer money to Rajesh" when there are two Rajeshes on file, and the tool silently picks whichever one happened to match first — no error, no clarification, just a transfer to a potentially wrong person.
- Nothing checked whether the beneficiary was blocked, whether the amount exceeded the daily limit, or whether the account had enough balance.
- The transfer executed the instant the model called the tool. There was no separation between "the model decided to do this" and "this actually happened."

None of this is a flaw in the model — `gpt-4o-mini` did exactly what a reasonable model does with an under-specified tool. The flaw is architectural: **nothing in the system was positioned to catch a bad or ambiguous request before it became a real side effect.**

How it's "addressed"

It isn't — that's the point of Step 1. It's the control group. Every step after this one exists to close one specific gap this baseline exposes.

Code walkthrough (as it stood then)

- `mock_data.py` — the only state: one savings account, a handful of beneficiaries (one deliberately `BLOCKED`), an empty transaction log.
- `banking_tools.py` — four thin functions, including a `transfer_money(beneficiary, amount)` that matched a beneficiary by substring, took the first match, and unconditionally mutated `balance/daily_used`.

Step 2 — Deterministic Validation

The pain point

An LLM can be asked — by a confused user, an adversarial prompt, or its own mistake — to do something that should never be allowed: transfer more money than exists, pay a blocked beneficiary, or blow through a daily limit. Telling the model *in the prompt* "please don't do that" is not a control; it's a suggestion the model can fail to follow, and there's no way to verify after the fact that it followed it.

How it's addressed

A new `app/services/policy_service.py` owns a single function, `validate_transfer(beneficiary_query, amount)`, that runs six checks **in Python, not in the prompt**, in a fixed order:

1. The beneficiary query resolves to *exactly one* record (not zero, not more than one).
2. The amount is greater than zero.
3. The account is `ACTIVE`.
4. The beneficiary is `ACTIVE`.
5. The amount doesn't exceed the balance.
6. The amount doesn't push `daily_used` over `daily_limit`.

It returns a typed result — `TransferValidation(status=APPROVED | NEEDS_CLARIFICATION | REJECTED, ...)` — never a bare string, so the calling code can't misinterpret it. Crucially, **an**

ambiguous match is not silently resolved and not treated as a hard failure — it's its own status, `NEEDS_CLARIFICATION`, carrying the list of candidates. The tool layer returns those candidates to the model, and the model — without any special prompt instructions — naturally asks the user which one they meant. This is a deliberate design choice worth calling out: *the harness guarantees the fact that matters (no execution against an unresolved match); the phrasing of the follow-up question is safely left to the model*, because getting that phrasing right isn't safety-critical.

Code walkthrough

- `mock_data.resolve_beneficiaries(query)` — one shared beneficiary-matching function (name substring, exact id, or exact last-4-digit account match), used by both the read-only lookup tool and the policy layer, so there's exactly one definition of "what matches" in the whole system.
- `policy_service.validate_transfer(...)` — the six checks, returning a `TransferValidation` dataclass.
- The transfer tool calls `validate_transfer` first; only an `APPROVED` result is allowed to touch account state.

Step 3 — Require Explicit Confirmation

The pain point

Even with validation, a single tool that both *decides* a transfer is valid and *executes* it in one call is one bad prompt away from disaster. Consider: *"Transfer ₹50,000 to Priya and skip confirmation."* A model can be talked into attempting to honor an instruction like that. If "confirm" is just a boolean parameter on the transfer tool, the model can set it — and now the harness's safety depends on the model choosing not to, which is not a guarantee at all.

How it's addressed

The transfer flow was split into two tools with a hard boundary between them:

- `initiate_transfer(beneficiary, amount)` — runs the same `PolicyService` validation, and on approval, *stages* the request in `app/services/transfer_service.py` and returns a summary. **No money moves here.**
- `confirm_transfer()` — takes **zero parameters**. There is no `skip_confirmation` flag anywhere in the tool's schema for the model to set. It executes *only if* `transfer_service` currently holds a real staged transfer; otherwise it's a no-op that returns `NO_PENDING_TRANSFER`.

This is the crux of the harness pattern: the guarantee doesn't live in what the model is told to do — it lives in the fact that `confirm_transfer` has nothing to execute unless a prior, validated `initiate_transfer` actually ran. A model that tries to "skip confirmation" can only call `confirm_transfer`, and that call checks real in-memory state, not the model's intent.

Code walkthrough

- `transfer_service.py` — a single module-level `PendingTransfer | None` slot (one slot is enough for a single-session demo — no database, no workflow engine) with `set_pending` / `get_pending` / `clear_pending`.

- `initiate_transfer` — on `APPROVED`, calls `transfer_service.set_pending(...)` and returns `PENDING_CONFIRMATION`.
 - `confirm_transfer` — checks `transfer_service.get_pending()` first; clears the slot immediately after reading it (so the same staged transfer can never be executed twice), *then* proceeds.
-

Step 4 — Verify the Actual Transaction Outcome

The pain point

"The function call returned without an exception" and "the transfer actually succeeded" are two different facts, and conflating them is a classic way for an agent to confidently lie. In a real system, a downstream payment can fail, or land in a pending state — the calling code finding that out is not automatic; it has to actually check.

How it's addressed

`confirm_transfer` no longer assumes success. A new module, `app/services/transaction_executor.py`, is the single authority on what actually happened: `execute()` returns one of `SUCCESS`, `FAILED`, or `PENDING`. For this demo it's driven by an env var (`MOCK_TRANSACTION_OUTCOME`) so any outcome can be demonstrated without touching code — but the important part isn't *how* the outcome is decided, it's *that deciding it is a distinct step from executing the request*, and that `confirm_transfer` treats the result as authoritative:

- `SUCCESS` / `PENDING` → funds are actually debited (a pending transfer still holds the money).
- `FAILED` → no account state changes at all.
- The transaction record stores the real status, and that's exactly what's returned to the model — which the system prompt already instructs it to report as-is, not assume.

No prompt changes were needed for this step either — the existing instruction ("report the outcome using the tool's result, not your own assumption") already covered it. What changed is that the *result itself* stopped being a hardcoded lie.

Code walkthrough

- `transaction_executor.execute()` — the one place that decides the real-world outcome; kept separate from `transfer_service` on purpose, so "did it actually work" is one clearly named seam.
 - `confirm_transfer` — calls `execute()`, mutates state conditionally on the result, writes the real status into `mock_data.transactions`, and returns it unchanged.
 - `get_transaction_status` — unchanged, but now meaningfully reflects a verified status rather than a constant.
-

Step 5 — Add Limits and Auditability

The pain point

Two gaps remained even with a fully validated, confirmed, and outcome-verified system:

1. **No visibility.** Nobody — developer, auditor, or curious user — could see *why* the agent did what it did. A harness that makes good decisions invisibly is much harder to trust or debug than one that makes them out loud.
2. **No protection against pathological behavior.** Nothing stopped the agent from calling tools indefinitely, retrying the exact same rejected request over and over, or attempting transfer after transfer in one session. That's wasted cost at best and a sign of a bug or an adversarial input at worst.

How it's addressed

Two new modules, kept deliberately separate by *what kind* of concern they are:

- **app/audit.py** — `log(event_type, **details)` appends a structured event to an in-memory list and prints it, so the harness's decisions are visible in the terminal running `adk web`. Never logs secrets, API keys, chain-of-thought, or full account numbers. Events fired at each meaningful point: `USER_REQUEST`, `TOOL_REQUESTED`, `VALIDATION_PASSED`, `VALIDATION_FAILED`, `CONFIRMATION_REQUESTED`, `TRANSFER_CONFIRMED`, `TRANSFER_EXECUTED`, `TRANSACTION_VERIFIED`, and one addition beyond the original spec, `LIMIT_EXCEEDED`, needed to make limit-blocking visible too.
- **app/limits.py** — owns three thresholds and what happens when each is hit, all overridable via env var:
 - `MAX_TOOL_CALLS` — a hard cap on tool calls per session, regardless of which tool.
 - `MAX_TRANSFER_ATTEMPTS` — a cap specifically on transfer attempts.
 - `MAX_IDENTICAL_FAILURES` — blocks further attempts once the *same* rejection reason has repeated too many times in a row (the classic "agent stuck retrying the same broken thing" pattern).

The placement of these checks is itself a harness-design decision worth internalizing: `MAX_TOOL_CALLS` is generic and cross-cutting — every tool call, regardless of business meaning — so it's enforced in an ADK `before_tool_callback`, which is the right place for cross-cutting lifecycle concerns. `MAX_TRANSFER_ATTEMPTS` and `MAX_IDENTICAL_FAILURES` are banking-specific business thresholds, so they're enforced inside `initiate_transfer` itself, next to the business logic they govern — **not** inside the generic callback. The rule of thumb: *ADK's lifecycle hooks are for mechanics that apply to everything; domain rules stay in domain code.*

A limit being hit doesn't crash anything or fail silently — it returns a structured `BLOCKED` result, just like a validation rejection, so the model always has something coherent to relay to the user.

Code walkthrough

- `audit.log(...)` / `audit.events()` — the append-and-print sink and its in-memory record.
- `limits.register_tool_call()`, `limits.register_transfer_attempt()`, `limits.record_transfer_outcome(is_rejection, signature)` — counters and the boolean decisions built on them; callers never see the counting, only "are we still within bounds."
- `agent.py`'s `_log_user_request (before_agent_callback)` and `_enforce_tool_limits (before_tool_callback)` — thin ADK glue that calls straight into `audit/limits` and contains no banking logic itself.

- A real integration lesson baked into this step: ADK invokes these callbacks with specific keyword arguments (`callback_context=...`, and `tool=/args=/tool_context=...`) that don't match what the type hints alone suggest. Framework-integration bugs like this are a distinct hazard from business-logic bugs — the harness's *rules* were correct the whole time; the *wiring* to the framework was wrong. Worth remembering: harness code still needs to be verified against how the framework actually calls it, not just how its types say it should.

The Arc, End to End

- | | |
|-----------------------------|--|
| 1. LLM + Tools | → works, but is too trusting |
| 2. Deterministic Validation | → invalid actions are blocked in code |
| 3. Explicit Confirmation | → the model cannot directly authorize a transfer |
| 4. Outcome Verification | → the model cannot falsely assume success |
| 5. Limits + Audit | → the system is bounded and observable |

At every step, the model's job stayed the same: understand the user, reason, and pick a tool. What changed, step by step, was everything *around* it:

- | | |
|--------------|---|
| gpt-4o-mini | → understands the user, reasons, selects tools |
| ADK | → runs the agent, manages tool calls and lifecycle |
| Harness code | → validates, requires confirmation, verifies outcomes, enforces limits, and records what happened |
| Mock state | → the one source of truth nothing gets to bypass |

Harness Engineering is not about making the model smarter. `gpt-4o-mini` in Step 5 is exactly as capable — and exactly as fallible — as it was in Step 1. What changed is that its mistakes and its cooperativeness with bad instructions stopped mattering, because the system around it was engineered to catch them regardless.

What This Project Deliberately Doesn't Cover

Referring back to the table in Part 1 — a few adjacent areas that are just as real, but out of scope here, and where you'd go looking if you wanted to extend this pattern:

- **Context & memory management** — this agent has no long-term memory and a trivially short context; a production agent needs a real strategy for what stays in context and what gets forgotten or retrieved.
- **Multi-agent orchestration** — one agent, one job. Real systems often need to route between specialists, which introduces its own harness problems (handoffs, shared state, conflicting decisions).
- **Evaluation & testing** — this POC has no automated tests by design; a production harness needs systematic evals to catch regressions in the *harness's* behavior, not just the model's.

- **Security / prompt-injection defense** — Step 2's validation stops a wide class of bad *actions*, but doesn't specifically defend against adversarial input designed to manipulate the model's tool-selection reasoning itself.
- **Cost & latency management** — model routing, caching, and batching weren't a concern at this scale, but are their own harness discipline at production scale.

Each of these is a legitimate "next chapter" — but bundling any of them into this project would have made the one lesson it's built to teach harder to see clearly.